

ING1312 – Programación

UNIDAD 2: *Estructura de Datos y Funciones*

Librería Turtle, Recursividad

Alexander Baumgartner
alexander.baumgartner@uoh.cl



Instituto de Ciencias de la Ingeniería
Universidad de O'Higgins

Hay que importar las librerías que se utiliza:

```
1 import turtle                # Importar la libreria
2
3 crush = turtle.Turtle()
4 # Has algo con la tortuga llamada crush
5 turtle.done()
```

```
1 import turtle as ttl        # Importar la libreria y darle un nombre
2
3 crush = ttl.Turtle()
4 # Has algo con la tortuga llamada crush
5 ttl.done()
```

```
1 from turtle import *        # Importar todas las funciones (menos favorable)
2
3 crush = Turtle()
4 # Has algo con la tortuga llamada crush
5 done()
```

- La tortuga lleva un lápiz trazando líneas (o puntos o círculos).
- Siempre apunta a una dirección en el espacio 2D.
- Al comienzo se encuentra en la posición (0, 0) y apunta hacia la derecha.
- Podemos mover la tortuga.

```
1 import turtle
2
3 crush = turtle.Turtle()      # Hay que construir una tortuga
4 crush.forward(100)           # Luego se puede mover y rotar
5 crush.right(90)
6 crush.fd(100)                # fd es una abreviatura de forward
7
8 turtle.done()
```

En la pantalla, por defecto, la tortuga está representada por una flecha.
Abreviaturas comunes: fd = forward, bk = backward, rt = right, lt = left

- Se puede cambiar el color del lápiz.
- Hay varios comandos para mover la tortuga.

```
1 import turtle
2
3 crush = turtle.Turtle()
4 crush.pencolor("blue")
5 crush.forward(100)
6 crush.left(90)
7 crush.pencolor("red")
8 crush.backward(100)      # la tortuga se puede mover hacia atras
9
10 turtle.done()
```

La tortuga puede levantar el lápiz.

```
1 import turtle
2
3 crush = turtle.Turtle()
4 crush.forward(100)
5 crush.right(90)
6 crush.penup()
7 crush.forward(100)
8 crush.pendown()
9 crush.right(90)
10 crush.forward(100)
11
12 turtle.done()
```

- Por defecto la tortuga comienza en (0, 0) y apunta hacia la derecha.
- Podemos mandar la tortuga a ir a un punto (x, y) en la pantalla.

```
1 import turtle
2
3 crush = turtle.Turtle()
4 crush.goto(100, 100) # Mandar Crush al punto (100, 100)
5 crush.right(90)
6 crush.forward(100)
7 crush.right(90)
8 crush.forward(100)
9
10 turtle.done()
```

- El modulo math contiene constantes y funciones matemáticas.
- Por ejemplo: e , π , $\sin(x)$, $\cos(x)$, $\tan(x)$, $\text{abs}(x)$, $\log(x)$, $\text{gcd}(x, y)$,...

```
1 import turtle
2 import math
3
4 turtle.setworldcoordinates(0, -1, 2 * math.pi, 1)      # llx , lly , urx , ury
5 bob = turtle.Turtle()
6 bob.goto(1/2 * math.pi, math.sin(1/2 * math.pi))
7 bob.goto(math.pi, math.sin(math.pi))
8 bob.goto(3/2 * math.pi, math.sin(3/2 * math.pi))
9 bob.goto(2 * math.pi, math.sin(2 * math.pi))
10
11 turtle.done()
```

Ojo: La tortuga se puede mover por afuera de la pantalla.

¡Autoestudio!

- Documentación online:

`https://docs.python.org/es/3/library/turtle.html`

`https://docs.python.org/es/3/library/math.html`

Ordene las siguientes líneas de código de tal manera que se ejecute correctamente.

```
1  alice.done()  
2  alice.forward(200)  
3  import turtle  
4  for i in range(10):  
5  alice = turtle.Turtle()  
6  alice.left(144)
```

Recursividad es una técnica de programación que tiene su fundamento en la **inducción matemática**.

Para que un problema pueda ser resuelto en forma recursiva, debe poder expresarse a partir de:

- Un **caso base**: es un caso para el cual la resolución es inmediata.
- Una **regla inductiva** que se utiliza a si misma (la recursión): es decir poder expresar la solución para este caso en base al mismo problema.

Definición de potencia: sea $a \in \mathbb{R}$ y sea $n \in \mathbb{N}$ entonces a^n simboliza multiplicar la base a por si misma n veces, o simplemente escribir:

$$a^n = \begin{cases} 1 & \text{si } n = 0, \\ a \cdot a^{n-1} & \text{si } n > 0. \end{cases}$$

caso base
regla inductiva

```
1 def potencia(a, n):  
2     """ Computa la potencia n del número a.  
3  
4     potencia: float int -> float  
5     ejemplo: potencia(3.3, 4) debe producir 118.5921 """  
6     if n == 0:  
7         return 1 # caso base  
8     else:  
9         return a * potencia(a, n - 1) # regla inductiva
```

Una función que se define en términos de sí mismo es llamada **función recursiva**.

Consideramos el ejemplo de la cuenta atrás que hemos visto antes:

```
1 print("Cuenta atrás")
2 a = 10
3 while a > 0 :
4     print(a)
5     a -= 1
```

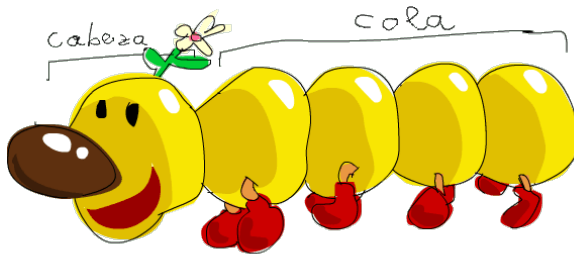
- El **caso base** es $a = 0$. En este ejemplo, el caso base no hace nada.
- La **regla inductiva** esta compuesta de los siguientes pasos: imprimir el valor y ejecutar la recursión con el valor decrementado.
- Aunque no devuelve ningún valor podemos escribir una función recursiva:

$$\text{cuentaAtras}(a) = \begin{cases} \text{no hacer nada} & \text{si } a = 0, \\ \text{imprimir } a \text{ y ejecutar } \text{cuentaAtras}(a - 1) & \text{si } a > 0. \end{cases}$$

```
1 def cuentaAtras(a):
2     if a > 0:
3         print(a)
4         cuentaAtras(a - 1)
```

A veces es conveniente ver a las listas como la unión de

- un elemento, llamado la cabeza, y
- otra lista, llamada la cola.



Definimos dos funciones: una para la cabeza y otra para la cola.

```
1 def cabeza(lista):  
2     return lista[0]  
3 def cola(lista):  
4     return lista[1:]
```

lista[1:len(lista)]

Usamos las dos funciones para **definir el máximo** de una lista **recursivo**.

```
1 def cabeza(lista):  
2     return lista[0]  
3 def cola(lista):  
4     return lista[1:] # lista[1:len(lista)]
```

- **Caso base:** El máximo de una lista que tiene un sólo elemento es ese elemento.
- **Regla inductiva:** El máximo de una lista es el mayor valor entre la cabeza y el máximo de su cola.

```
1 def maximo(lista):  
2     #caso base  
3     if len(lista) == 1:  
4         return cabeza(lista)  
5     #regla inductiva  
6     maximoCola = maximo(cola(lista))  
7     if cabeza(lista) > maximoCola:  
8         return cabeza(lista)  
9     return maximoCola
```

Mapear: aplicar una función dada a cada elemento de una lista.

El método mapear es levemente más complejo que los anteriores porque necesitamos ir **generando una lista** a medida que ejecutamos los casos recursivos.

- **Caso base:** el "map" de una lista vacía es otra lista vacía.
- **Caso inductivo:** el "map" de una lista es otra lista que se forma al concatenar el resultado de aplicar la función dada a la cabeza junto con la lista "mapeada" de la cola.

```
1 def mapear(lista, fnc):  
2     if len(lista) == 0:  
3         return []  
4     return [fnc(cabeza(lista))] + mapear(cola(lista), fnc)
```

Aplicar mapear:

```
1 def disminuir(n):  
2     return n - 1
```

Python ofrece la posibilidad de definir funciones λ .

```
1 disminuir = lambda n : n - 1
```

```
1 >>> mapear([1, 2, 5, -2, 13], disminuir)  
2 [0, 1, 4, -3, 12]
```



Generar de manera recursiva un string que contiene los números enteros a partir de 1 y hasta llegar a n (dado por el usuario) en forma inversa y separados por un espacio.

Para entender el concepto, notemos lo siguiente

- Si el usuario ingresa 1 entonces en pantalla se muestra el string '1'.
- El string mostrado para un n cualquiera resulta sera ' $n \ n - 1 \ \dots \ 2 \ 1$ '.



Escribir una función recursiva que reciba un número entero positivo y devuelva su factorial.



Calcular la sumatoria de una lista de forma recursiva:

- La sumatoria de una lista vacía es 0.
- La sumatoria de una lista es la cabeza más la sumatoria de la cola.



Escribir un programa que le pida al usuario un número entero n .
Luego imprime los n primeros números de la serie de Fibonacci.

La serie de Fibonacci es la siguiente sucesión infinita de números naturales:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597 ...

Los números F_i , $i \geq 0$ de la serie quedan definidos por las ecuaciones

$$F_0 = 0,$$

$$F_1 = 1,$$

$$F_n = F_{n-1} + F_{n-2}.$$



Escribir una función que reciba una muestra de números en una lista y devuelva otra lista con sus cuadrados. No se permite usar bucles.



¿Cuál es el resultado de ejecutar el siguiente programa?

```
1 def suma_mas_uno(n):  
2     if n == 0:  
3         return 1  
4     return n + suma_mas_uno(n - 1)  
5  
6 print(suma_mas_uno(4))
```

¿Que pasa si se cambia `if n == 0:` por `if n == 1:`?

¿Que pasa si se quita el caso base?



Analice la siguiente regla recursiva y escriba el programa correspondiente. ¿Qué hace la función?:

$$\textit{repetir}(a,b) = \begin{cases} 0, & \text{si } b = 0 \\ a, & \text{si } b = 1 \\ a + \textit{repetir}(a, b - 1), & \text{si } b > 1 \end{cases}$$



Escribir una función que toma un número entero e imprime el siguiente patrón, utilizando una recursión:

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
...
```

Como se puede cambiar el orden de imprimir las líneas? Es decir, como imprimir lo al revés, comenzando en 5 5 5 5 5, dado el número ingresado es 5?



¿Qué hace el código?

```
1 def function(y):  
2     if len(y) == 1:  
3         return "z"  
4     else:  
5         return "z"+function(y[1:])  
6 x = function("aaa")  
7 print(x)
```




Escriba una función recursiva que reciba una palabra y entregue como resultado la palabra invertida.



Dado el siguiente programa, diseñe una recursión que corresponde al bucle que computa la suma, y da el código Python de la función recursiva:

```
1  n = int(input('Da un numero: '))
2  suma = 0
3  while n > 1:
4      if n % 3 == 0:
5          suma += n
6      n -= 1
7  print(suma)
```

Para diseñar la recursión, usted debe identificar el **caso base** y lo que pasa en el **caso inductivo**.



Un palíndromo es una palabra que se lee igual adelante que atrás. Las siguientes palabras son ejemplos de palíndromos: ojo, oso, rotor, salas.

- Dar una función recursiva `es_palindromo` para determinar si se trata de un palíndromo.
- Escribir un programa que permita al usuario ingresar una palabra e identifica si la palabra dada es un palíndromo.

Por ejemplo:

- Si el usuario ingresa la palabra "oso" el programa debe imprimir: Si, oso es un palíndromo.
- Si se ingresa la palabra "sol" el programa debe imprimir: No, sol no es un palíndromo.